CSE103: Structured Programming

Lecture 11

Arrays

- Advantage: store large set of data & manipulate them under one name.
- Disadvantage:
  - all the elements stored in an array should be of the <u>same data type</u>.
  - If we need to use a collection of different data type items it is not possible using an array.

Structures

- **struct** is used to declare a new data-type.
- A struct is heterogeneous in that it can be composed of data of different types.
- Basically this means grouping variables together.
- These variables can be of different types, and each has a name which is used to select it from the structure.

Structures

- **For example:**

Name	Batch	Roll	Incourse 1	Incourse 2	Total
Joe	18	101	55	89	144
Karim	18	103	67.5	100	167.5
Rahim	18	104	40	70	110
Uday	18	10	50	60	110
Gourav	18	55	100	100	200
Zadu	18	68	3	80	83
Modhu	18	33	45	69	114
..					
…					

Can you solve it using Array?

- **We want to print the merit list of this class!**
- **Descending order of total scores!**

Structures are there to rescue you!

- In each row (each record) there are five different types of information.
  - Name
  - Batch
  - Roll
  - Incourse 1 mark
  - Incourse 2 mark
  - Total mark

Structures are there to rescue you!

- In each row (each record) there are five different types of information.
  - Name (char name[80])
  - Batch (int batch)
  - Roll (int roll)
  - Incourse 1 mark (double incourse1)
  - Incourse 2 mark (double incourse2)
  - Total mark (double total)

Defining the record/structure


```
struct Student{
    char name[80];
    int batch;
    int roll;
    double incourse1;
    double incourse2;
    double total;
};
```

Name of the type

Members of the **struct**

Defining the record/structure


```
#include<stdio.h>

struct Student{
    char name[80];
    int batch;
    int roll;
    double incourse1;
    double incourse2;
    double total;
};

int main(){
    return 0;
}
```

Declaring a variable of type "Student"


```
#include<stdio.h>

struct Student{
    char name[80];
    int batch;
    int roll;
    double incourse1;
    double incourse2;
    double total;
};

int main(){
    struct Student s1;
    return 0;
}
```

Address	Content	Child	Parent
4000	??	name	s1
4100	??	batch	
4104	??	roll	
4108	??	incourse1	
4116	??	incourse2	
4124	??	total	

Declaring a variable of type "Student"


```
#include<stdio.h>

struct Student{
    char name[80];
    int batch;
    int roll;
    double incourse1;
    double incourse2;
    double total;
};

int main(){
    struct Student s1;

printf("%d\n",sizeof(s1));
    return 0;
}
```

Output: 112

Address	Content	Child	Parent
4000	??	name	s1
4100	??	batch	
4104	??	roll	
4108	??	incourse1	
4116	??	incourse2	
4124	??	total	

Declaring a variable of type "Student"


```
#include<stdio.h>

struct Student{
    char name[80];
    int batch;
    int roll;
    double incourse1;
    double incourse2;
    double total;
};

int main(){
    struct Student s1,s2;
    return 0;
}
```

Address	Content	Child	Parent
4000	??	name	s1
4100	??	batch	
4104	??	roll	
4108	??	incourse1	
4116	??	incourse2	
4124	??	total	
8000	??	name	s2
8100	??	batch	
8104	??	roll	
8108	??	incourse1	
8116	??	incourse2	
8124	??	total	

Declaring a variable of type "Student"


```
#include<stdio.h>

struct Student{
    char name[80];
    int batch;
    int roll;
    double incourse1;
    double incourse2;
    double total;
}s1, s2; //Global variable

int main(){
    return 0;
}
```

Address	Content	Child	Parent
4000	??	name	s1
4100	??	batch	
4104	??	roll	
4108	??	incourse1	
4116	??	incourse2	
4124	??	total	
8000	??	name	s2
8100	??	batch	
8104	??	roll	
8108	??	incourse1	
8116	??	incourse2	
8124	??	total	

Member accessing within a structured variable Using DOT (.) operator

```
#include<stdio.h>
struct Student{
    char name[80];
    int batch;
    int roll;
    double incourse1;
    double incourse2;
    double total;
};
int main(){
    struct Student s1,s2;
    s1.batch=16;
    s1.roll=55;
    s1.incourse1=67.8;
    s1.incourse2=68.0;
    s1.total = s1.incourse1+s1.incourse2;
    gets(s1.name);
    return 0;
}
```

Address	Content	Child	Parent
4000	Rahim	name	s1
4100	16	batch	
4104	55	roll	
4108	67.8	incourse1	
4116	68.0	incourse2	
4124	135.8	total	
8000	??	name	s2
8100	??	batch	
8104	??	roll	
8108	??	incourse1	
8116	??	incourse2	
8124	??	total	

Initializer list of a structure variable


```
#include<stdio.h>
struct Student{
    char name[80];
    int batch;
    int roll;
    double incourse1;
    double incourse2;
    double total;
};
int main(){
    struct Student s2;
    struct Student s1=
        {"Karim",16,58,3.0,88.5,91.5};
    return 0;
}
```

Address	Content	Child	Parent
4000	Karim	name	s1
4100	16	batch	
4104	58	roll	
4108	3.0	incourse1	
4116	88.5	incourse2	
4124	91.5	total	
8000	??	name	s2
8100	??	batch	
8104	??	roll	
8108	??	incourse1	
8116	??	incourse2	
8124	??	total	

Operations on Structure variables: Assignment


```
#include<stdio.h>
struct Student{
    char name[80];
    int batch;
    int roll;
    double incourse1;
    double incourse2;
    double total;
};
int main(){
    struct Student s2;
    struct Student s1=
        {"Karim",16,58,3.0,88.5,91.5};
    s2 = s1;
    return 0;
}
```

Address	Content	Child	Parent
4000	Karim	name	s1
4100	16	batch	
4104	58	roll	
4108	3.0	incourse1	
4116	88.5	incourse2	
4124	91.5	total	
8000	Karim	name	s2
8100	16	batch	
8104	58	roll	
8108	3.0	incourse1	
8116	88.5	incourse2	
8124	91.5	total	

Operations on Structure variables: Call by value


```
#include<stdio.h>
struct Student{
    char name[80]; int batch; int roll; double incourse1;
    double incourse2; double total;
};

void printAStudent( struct Student p ){
    printf("Name: %s\n",p.name);
    printf("Batch: %d\n",p.batch);
    printf("Roll: %d\n",p.roll);
    printf("Incourse 1: %lf\n",p.incourse1);
    printf("Incourse 2: %lf\n",p.incourse2);
    printf("Total: %lf\n",p.total);
}
int main(){
    struct Student s2, s1= {"Karim",16,5
    s2 = s1;
    printAStudent(s2);
    return 0;
}
```

```
Name: Karim
Batch: 16
Roll: 58
Incourse 1: 3.000000
Incourse 2: 88.500000
Total: 91.500000
```

Operations on Structure variables: Call by Reference


```
#include<stdio.h>
struct Student{
    char name[80]; int batch; int roll; double incourse1;
    double incourse2; double total;
};
void printAStudent( struct Student *p ){
    printf("Name: %s\n", (*p).name );
    printf("Batch: %d\n", (*p).batch);
    printf("Roll: %d\n", (*p).roll);
    printf("Incourse 1: %lf\n", (*p).incourse1 );
    printf("Incourse 2: %lf\n", (*p).incourse2 );
    printf("Total: %lf\n", (*p).total );
}
int main(){
    struct Student s2, s1= {"Karim",16,58,3.0,88.5,91.5};
    s2 = s1;
    printAStudent(&s2);
    return 0;
}
```

Operations on Structure variables: Call by Reference


```
#include<stdio.h>
struct Student{
    char name[80]; int batch; int roll; double incourse1;
    double incourse2; double total;
};
void printAStudent( struct Student *p ){
    printf("Name: %s\n", p->name );
    printf("Batch: %d\n", p->batch);
    printf("Roll: %d\n", p->roll);
    printf("Incourse 1: %lf\n", p->incourse1 );
    printf("Incourse 2: %lf\n", p->incourse2 );
    printf("Total: %lf\n", p->total );
}
int main(){
    struct Student s2, s1= {"Karim",16,58,3.0,88.5,91.5};
    s2 = s1;
    printAStudent(&s2);
    return 0;
}
```

Array of structure variables


```
#include<stdio.h>
#include<conio.h>

struct Student{
    char name[80];
    int batch;
    int roll;
    double incourse1;
    double incourse2;
    double total;
};
int main(){
    struct Student s[50];

    return 0;
}
```

Address	Content	Child	Parent
4000	??	name	s1
4100	??	batch	
4104	??	roll	
4108	??	incourse1	
4116	??	incourse2	
4124	??	total	
4132	??	name	s2
4231	??	batch	
4235	??	roll	
4239	??	incourse1	
4247	??	incourse2	
4255	??	total	

THANK
YOU!

QUESTIONS?
COMMENTS?
CONCERNS?